Spatial Variation in Violent Crime and Socioeconomic Deprivation in London

Preparation

- **GitHub / data repository:** `https://github.com/ZLH2-2713/Data-Science-for-Spatial-Systems-DSSS-CASA0006-.git`
- **Approximate word count (markdown narrative):** 1,453 words
- **Runtime:** 3 minutes

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and discussion
6. Conclusion
7. References

Introduction

Violence is a major urban policy issue because it is unevenly distributed across neighbourhoods and often linked to wider social disadvantage. Crime geography research has long argued that crime is not randomly scattered in space. Shaw and McKay (1942) showed that neighbourhood social conditions shape delinquency, while Sampson, Raudenbush and Earls (1997) found that concentrated disadvantage and weak collective efficacy are associated with violent crime. Spatial studies also stress clustering, meaning that nearby places often share similar risk profiles (Anselin, 1995; Chainey and Ratcliffe, 2005).

This notebook focuses on **Violence Against the Person** at the London **LSOA** level rather than combining all offence types. That decision matters because different crimes have different mechanisms, and a total-crime measure can hide meaningful relationships. Following the assessment guidance, I use a **rate** rather than a raw count by calculating incidents per 1,000 residents. This reduces scale bias caused by uneven population size across neighbourhoods.

The study combines Metropolitan Police crime records with the English Indices of Deprivation and population denominators. To avoid endogeneity, crime-related deprivation variables are excluded from the predictors. The aim is to test whether dimensions such as income, employment, education, health and housing barriers are

associated with local violence rates, and whether machine-learning models capture patterns that a linear model misses.

Research Questions

1. **Which dimensions of socioeconomic deprivation are most strongly associated with violence against the person rates across London LSOAs?**
2. **Do machine-learning models predict neighbourhood violence rates better than interpretable linear benchmarks?**
3. **Do the observed patterns support the wider expectation that concentrated disadvantage is linked to higher urban violence?**

Data

This study integrates four primary datasets at the 2021 LSOA (Lower Layer Super Output Area) level to analyze the spatial relationship between violent crime and socioeconomic deprivation in London.

1. Violent Crime Records Source: Metropolitan Police Service (MPS).

Description: Monthly incident counts for "Violence Against the Person" (including injury, non-injury, and harassment).

Role: The dependent variable, aggregated and normalized by population to calculate average annual crime rates.

2. Index of Multiple Deprivation (IoD 2025) Source: Department for Levelling Up, Housing and Communities (DLUHC).

Description: Domain-specific scores for multidimensional poverty, including Income, Employment, Education, and Health.

Role: The primary independent variables used to measure concentrated disadvantage.

3. Population Estimates (Mid-2024) Source: Office for National Statistics (ONS).

Description: Mid-year population counts disaggregated by age and sex for each LSOA.

Role: Provides the denominators for crime rate standardization and demographic control variables (e.g., proportion of young adults).

4. LSOA Boundary Files Source: ONS Geography Portal.

Description: Digital vector boundaries (Shapefiles) for London’s 2021 LSOA geography.

Role: The spatial framework for data integration, spatial weight matrix construction, and GIS visualization.

Dataset	Source	Spatial Resolution	Temporal Scope	Key Variables
Violent Crime Records	Metropolitan Police Service (MPS)	LSOA (2021)	2024.03 - 2026.03	Violence Against the Person (Counts/Rates)
Index of Multiple Deprivation	DLUHC (IoD 2025)	LSOA (2021)	2025	IMD Scores, Income, Employment, Education
Population Estimates	ONS (Mid-2024)	LSOA (2021)	Mid-2024	Total Population, Age-Sex Structure
LSOA Boundary Files	ONS Geography Portal	LSOA (2021)	2021 Census	Shapefiles, GeoJSON, LSOA21CD

In [1]:
```
# Install required packages automatically if missing (the first run may t
import sys
import subprocess
import importlib

required = {
    'pandas': 'pandas',
    'numpy': 'numpy',
    'matplotlib': 'matplotlib',
    'seaborn': 'seaborn',
    'sklearn': 'scikit-learn',
    'statsmodels': 'statsmodels'
}

for mod, pkg in required.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', pk

print('All required packages are ready.')
```

```
All required packages are ready.
```

In [2]:
```
import warnings
warnings.filterwarnings('ignore')

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LassoCV
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_s

import statsmodels.api as sm

plt.style.use('seaborn-v0_8-whitegrid')
pd.set_option('display.max_columns', 200)

# Global image output directory
IMG_DIR = 'image'
os.makedirs(IMG_DIR, exist_ok=True)
print(f'Images will be saved to: {IMG_DIR}')

# Automatically save the current figure each time plt.show() is called
_auto_fig_counter = {'n': 1}
_original_show = plt.show

def _show_and_save(*args, **kwargs):
    fig = plt.gcf()
    if fig is not None and len(fig.get_axes()) > 0:
        fname = f"figure_{_auto_fig_counter['n']:03d}.png"
        fig.savefig(os.path.join(IMG_DIR, fname), dpi=300, bbox_inches='t
        _auto_fig_counter['n'] += 1
    return _original_show(*args, **kwargs)

plt.show = _show_and_save
```

```
Images will be saved to: image
```

In [3]:

```
# Check and install spatial-analysis libraries
spatial_libs = {
    'geopandas': 'geopandas',
    'libpysal': 'libpysal',
    'esda': 'esda',
    'mgwr': 'mgwr',
    'shap': 'shap'
}

for mod, pkg in spatial_libs.items():
    try:
        importlib.import_module(mod)
        print(f'{pkg} already installed')
    except ImportError:
        print(f'Installing {pkg}...')
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', pk
        print(f'{pkg} installed successfully')

# Import spatial-analysis libraries
from libpysal.weights import Queen
from libpysal import weights
from esda.moran import Moran, Moran_Local
import geopandas as gpd
try:
    from mgwr.gwr import GWR, MGWR
    from mgwr.sel_bw import Sel_BW
    print('MGWR imported successfully')
except:
    print('MGWR not available, alternative methods will be used')

print('Spatial analysis libraries are ready.')
```

```
geopandas already installed
libpysal already installed
esda already installed
mgwr already installed
shap already installed
MGWR imported successfully
Spatial analysis libraries are ready.
```

Methodology

The notebook is organised around four main methods directly related to the research question. **First**, data preprocessing constructs a comparable rate-based outcome and merges crime, deprivation and population data. **Second**, **OLS regression** provides an interpretable baseline. **Third**, **LassoCV** is used as a regularised linear model to shrink weak predictors. **Fourth**, **Random Forest** captures non-linear relationships and generates feature-importance rankings.

Several supporting diagnostics are retained. ESDA tests spatial clustering, cross-validation checks model stability, and residual diagnostics, bootstrap importance, SHAP and sensitivity analysis are used as supplementary interpretation tools. The **core analytical models** are OLS, LassoCV and Random Forest (Chainey and Ratcliffe, 2005).

In [4]:
```
# 1) Read the data
from pathlib import Path

# Support running the notebook from either the project root or the assign
base = Path.cwd()
data_dir = base / 'data' if (base / 'data').exists() else base.parent / '

crime_path = data_dir / 'MPS LSOA Level Crime (most recent 24 months).csv
imd_path = data_dir / 'File_7_IoD2025_All_Ranks_Scores_Deciles_Population

crime = pd.read_csv(crime_path)
imd = pd.read_csv(imd_path)

print('data_dir:', data_dir.resolve())
print('crime shape:', crime.shape)
print('imd shape:', imd.shape)
print('crime columns:', list(crime.columns[:8]), '...')
print('imd columns:', list(imd.columns[:8]), '...')
```

```
data_dir: /Users/hhj/Desktop/term2/zy2/data
crime shape: (105646, 30)
imd shape: (33755, 56)
crime columns: ['LSOA Code', 'LSOA Name', 'Borough', 'Major Category', 'Mi
nor Category', '202403', '202404', '202405'] ...
imd columns: ['LSOA code (2021)', 'LSOA name (2021)', 'Local Authority Dis
trict code (2024)', 'Local Authority District name (2024)', 'Index of Mult
iple Deprivation (IMD) Score', 'Index of Multiple Deprivation (IMD) Rank
(where 1 is most deprived)', 'Index of Multiple Deprivation (IMD) Decile
(where 1 is most deprived 10% of LSOAs)', 'Income Score (rate)'] ...
```

In [5]:
```
# 2) Focus on the crime type: Violence Against the Person
target_major = 'VIOLENCE AGAINST THE PERSON'
month_cols = [c for c in crime.columns if re.fullmatch(r"\d{6}", str(c))]
```

```
month_cols = sorted(month_cols)
last12 = month_cols[-12:]

crime_focus = crime.loc[crime['Major Category'] == target_major].copy()
crime_focus['crime_12m_count'] = crime_focus[last12].sum(axis=1)

crime_lsoa = (
    crime_focus.groupby('LSOA Code', as_index=False)['crime_12m_count']
    .sum()
)

print('Selected months:', last12[0], 'to', last12[-1])
print('Focused major category rows:', len(crime_focus))
print('LSOA aggregated rows:', len(crime_lsoa))
crime_lsoa.head()
```

```
Selected months: 202504 to 202603
Focused major category rows: 15371
LSOA aggregated rows: 4988
```

Out[5]:

	LSOA Code	crime_12m_count
0	E01000006	35
1	E01000007	208
2	E01000008	54
3	E01000009	81
4	E01000011	64

In [6]:

```
# 3) Select IMD features and avoid endogeneity
# Do not use the overall IMD score or crime-related fields to avoid predi
feature_map = {
    'Income Score (rate)': 'income_score',
    'Employment Score (rate)': 'employment_score',
    'Education, Skills and Training Score': 'education_score',
    'Health Deprivation and Disability Score': 'health_score',
    'Barriers to Housing and Services Score': 'barriers_score',
    'Living Environment Score': 'living_environment_score',
    'Income Deprivation Affecting Children Index (IDACI) Score (rate)': '
    'Income Deprivation Affecting Older People (IDAOPI) Score (rate)': 'i
    'Children and Young People Sub-domain Score': 'children_sub_score',
    'Adult Skills Sub-domain Score': 'adult_skills_score',
    'Outdoors Sub-domain Score': 'outdoors_score',
    'Total population: mid 2022': 'population_2022',
    'Dependent Children aged 0-15: mid 2022': 'children_0_15',
    'Older population aged 60 and over: mid 2022': 'older_60_plus',
    'Working age population 18-66 (for use with Employment Deprivation Do
}

imd_sub = imd[['LSOA code (2021)'] + list(feature_map.keys())].rename(col
imd_sub = imd_sub.rename(columns={'LSOA code (2021)': 'LSOA Code'})

# Derive demographic structure variables
imd_sub['child_share'] = imd_sub['children_0_15'] / imd_sub['population_2
imd_sub['older_share'] = imd_sub['older_60_plus'] / imd_sub['population_2
imd_sub['working_share'] = imd_sub['working_age_18_66'] / imd_sub['popula

df = crime_lsoa.merge(imd_sub, on='LSOA Code', how='inner')
```

```
df['violent_crime_rate_per_1000'] = df['crime_12m_count'] / df['populatio

df = df.replace([np.inf, -np.inf], np.nan).dropna()
print('Modeling dataframe shape:', df.shape)
df[['LSOA Code','crime_12m_count','population_2022','violent_crime_rate_p
```

```
Modeling dataframe shape: (4988, 21)
```

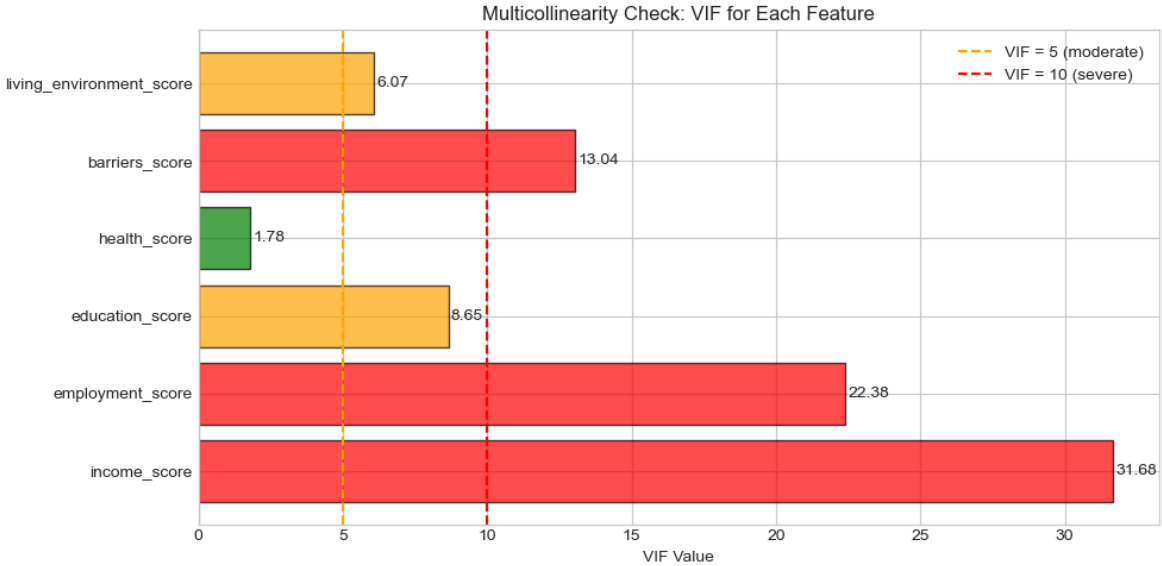
Out[6]:

	LSOA Code	crime_12m_count	population_2022	violent_crime_rate_per_1000
0	E01000006	35	1837	19.052803
1	E01000007	208	2961	70.246538
2	E01000008	54	1761	30.664395
3	E01000009	81	1859	43.571813
4	E01000011	64	1719	37.230948

In [7]:

```
# 4) Multicollinearity check (VIF) for model robustness
from statsmodels.stats.outliers_influence import variance_inflation_facto

# Features included in the VIF check
features_for_vif = ['income_score', 'employment_score', 'education_score'
                    'health_score', 'barriers_score', 'living_environment
X_vif = df[features_for_vif].dropna()

vif_data = pd.DataFrame()
vif_data['feature'] = features_for_vif
vif_data['VIF'] = [variance_inflation_factor(X_vif.values, i) for i in ra

print('=== Variance Inflation Factor (VIF) ===')
print(vif_data)
print('\nNote: VIF > 5 indicates moderate multicollinearity, VIF > 10 ind

# Visualise VIF values
fig, ax = plt.subplots(figsize=(10, 5))
colors = ['green' if v < 5 else 'orange' if v < 10 else 'red' for v in vi
bars = ax.barh(vif_data['feature'], vif_data['VIF'], color=colors, alpha=
ax.axvline(x=5, color='orange', linestyle='--', label='VIF = 5 (moderate)
ax.axvline(x=10, color='red', linestyle='--', label='VIF = 10 (severe)')
ax.set_xlabel('VIF Value')
ax.set_title('Multicollinearity Check: VIF for Each Feature')
ax.legend()
for bar, vif in zip(bars, vif_data['VIF']):
    ax.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height()/2,
            f'{vif:.2f}', va='center', fontsize=10)
plt.tight_layout()
plt.show()
```

```
=== Variance Inflation Factor (VIF) ===
                    feature        VIF
0              income_score  31.678182
1          employment_score  22.382492
2           education_score   8.646874
3              health_score   1.784368
4            barriers_score  13.035225
5  living_environment_score   6.066053

Note: VIF > 5 indicates moderate multicollinearity, VIF > 10 indicates sev
ere multicollinearity.
```



Descriptive statistics and variable distributions

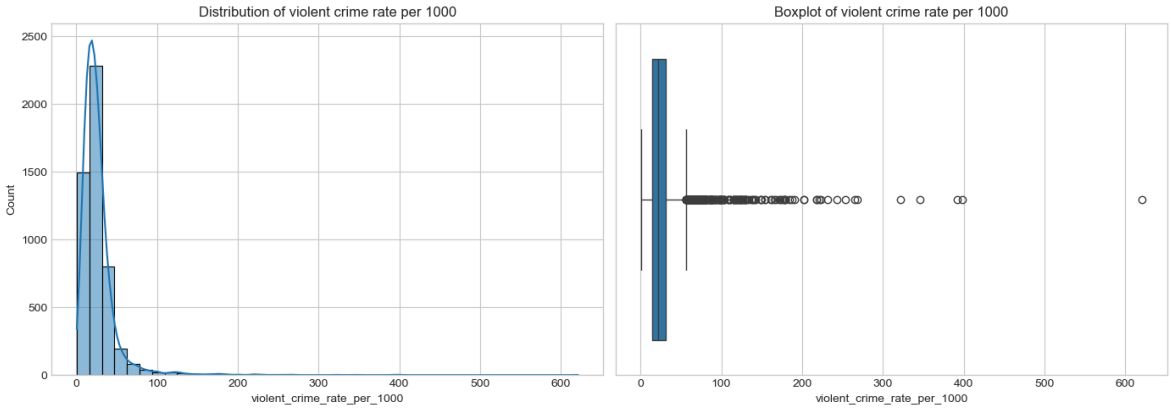
Before modelling, descriptive analysis is used to assess whether the outcome and predictors are plausible and sufficiently variable across space. Crime data are often skewed, with a small number of neighbourhoods accounting for a disproportionate share of incidents. Summaries and visualisations therefore help identify outliers, asymmetry and broad deprivation gradients before moving to inferential analysis.

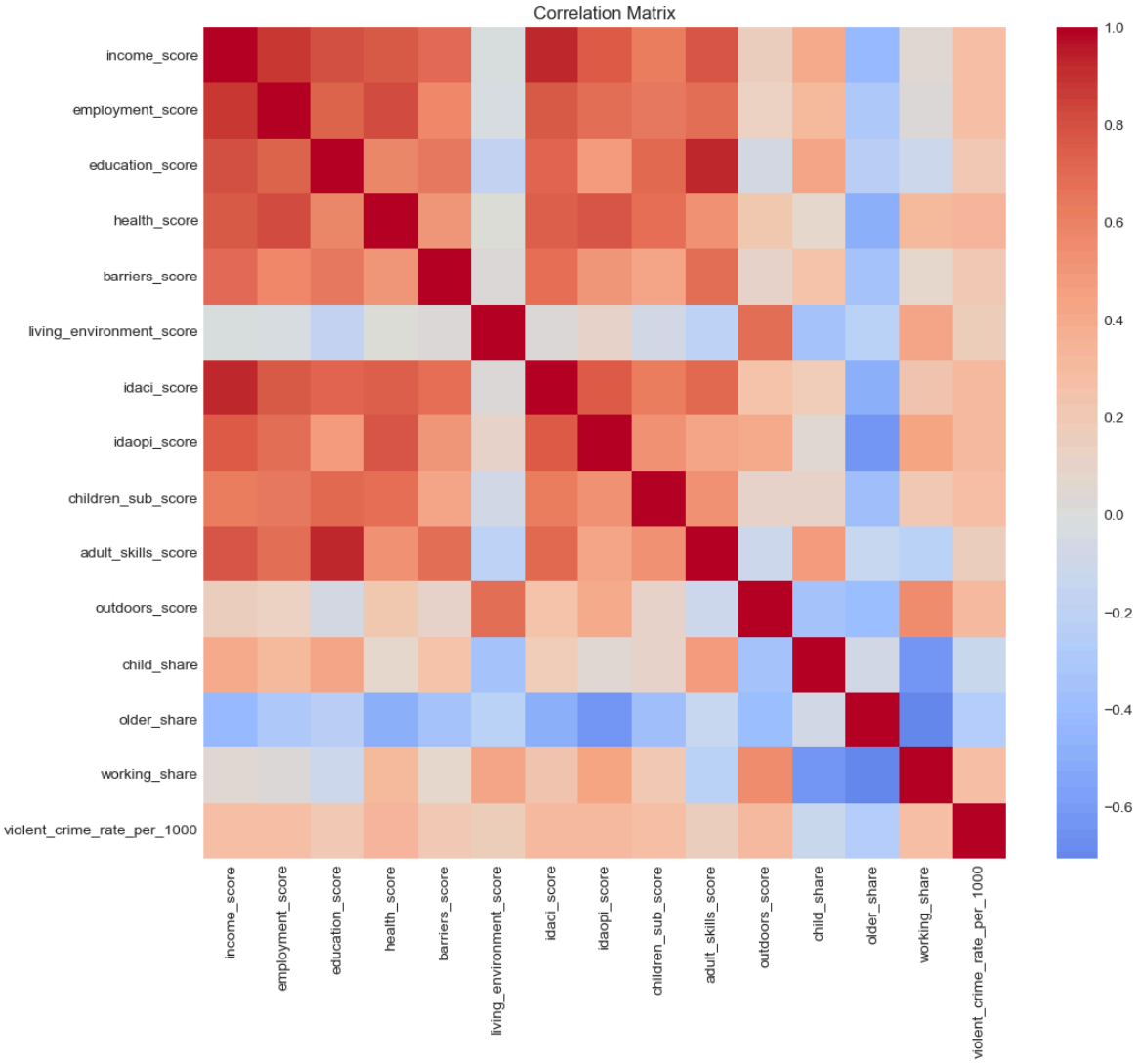
In [8]:
```
# Detailed descriptive statistics
print('='*60)
print('DATA DESCRIPTIVE STATISTICS')
print('='*60)
print(f'\nSample size: {len(df)} LSOAs')
print(f'Total violent crimes: {df["crime_12m_count"].sum():,}')
print(f'Average crime rate: {df["violent_crime_rate_per_1000"].mean():.2f
```

```
============================================================
DATA DESCRIPTIVE STATISTICS
============================================================

Sample size: 4988 LSOAs
Total violent crimes: 242,446
Average crime rate: 26.68 per 1,000
```

In [9]:
```
# 4) EDA: target distribution and correlations
target = 'violent_crime_rate_per_1000'

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(df[target], bins=40, kde=True, ax=axes[0])
```

```
axes[0].set_title('Distribution of violent crime rate per 1000')

sns.boxplot(x=df[target], ax=axes[1])
axes[1].set_title('Boxplot of violent crime rate per 1000')
plt.tight_layout()
plt.show()

corr_cols = [
    'income_score','employment_score','education_score','health_score',
    'barriers_score','living_environment_score','idaci_score','idaopi_sco
    'children_sub_score','adult_skills_score','outdoors_score',
    'child_share','older_share','working_share', target
]

plt.figure(figsize=(12, 10))
sns.heatmap(df[corr_cols].corr(), cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()
```





Exploratory Spatial Data Analysis (ESDA)

A core idea in spatial analysis is that nearby observations may not be independent. In crime research, neighbouring areas can share land use, accessibility, service provision, policing environments and social disadvantage, creating clustered outcomes. ESDA is therefore used here as a diagnostic step to establish whether violent crime rates display spatial structure.

The notebook uses **Global Moran's I** to test whether violence rates are more clustered than would be expected under spatial randomness, and **local Moran / LISA** to identify hot spots and cold spots. A positive Moran's I suggests that high-violence areas tend to be located near other high-violence areas. This does not prove causation, but it supports a place-based interpretation.

In [10]:
```
# Spatial autocorrelation analysis

# Create a distance-based spatial weight matrix
# Because true geometry is not used here, a KNN weight matrix is used as
from libpysal.weights import KNN

print('=== Global Spatial Autocorrelation Analysis ===')
```

```
# Build KNN weights using socioeconomic position as a proxy space
coords = df[['income_score', 'employment_score']].values
w_knn = KNN.from_array(coords, k=8)  # 8 nearest neighbours
w_knn.transform = 'r'  # Row-standardised

# Compute Moran's I
y_spatial = df['violent_crime_rate_per_1000'].values
moran = Moran(y_spatial, w_knn)

print(f"Moran's I: {moran.I:.4f}")
print(f"Expected I: {moran.EI:.4f}")
print(f"Z-score: {moran.z_norm:.4f}")
print(f"P-value: {moran.p_norm:.6f}")

# Interpret the result
if moran.p_norm < 0.001:
    significance = 'highly significant (p < 0.001)'
elif moran.p_norm < 0.01:
    significance = 'very significant (p < 0.01)'
elif moran.p_norm < 0.05:
    significance = 'significant (p < 0.05)'
else:
    significance = 'not significant'

print(f"\nInterpretation: The spatial autocorrelation is {significance}."
if moran.I > 0:
    print("Positive Moran's I indicates spatial clustering: high-crime ar
elif moran.I < 0:
    print("Negative Moran's I indicates spatial dispersion: high-crime ar
```

```
=== Global Spatial Autocorrelation Analysis ===
Moran's I: 0.0943
Expected I: -0.0002
Z-score: 14.0301
P-value: 0.000000

Interpretation: The spatial autocorrelation is highly significant (p < 0.0
01).
Positive Moran's I indicates spatial clustering: high-crime areas tend to
be near other high-crime areas.
```

In [11]:

```
# LISA cluster analysis (local hot-spot detection)
moran_local = Moran_Local(y_spatial, w_knn)

# Cluster label meanings:
# 1: HL (High-Low) - a high value surrounded by low values
# 2: LH (Low-High) - a low value surrounded by high values
# 3: LL (Low-Low) - cold spot cluster
# 4: HH (High-High) - hot spot cluster
cluster_labels = {0: 'Not Significant', 1: 'HL (Outlier)', 2: 'LH (Outlie
                  3: 'LL (Cold Spot)', 4: 'HH (Hot Spot)'}

df['lisa_cluster'] = moran_local.q
df['lisa_pvalue'] = moran_local.p_sim

# Count each cluster type
cluster_counts = pd.Series(moran_local.q).value_counts().sort_index()
print('=== LISA Cluster Distribution ===')
for cid in sorted(cluster_labels.keys()):
    count = cluster_counts.get(cid, 0)
```

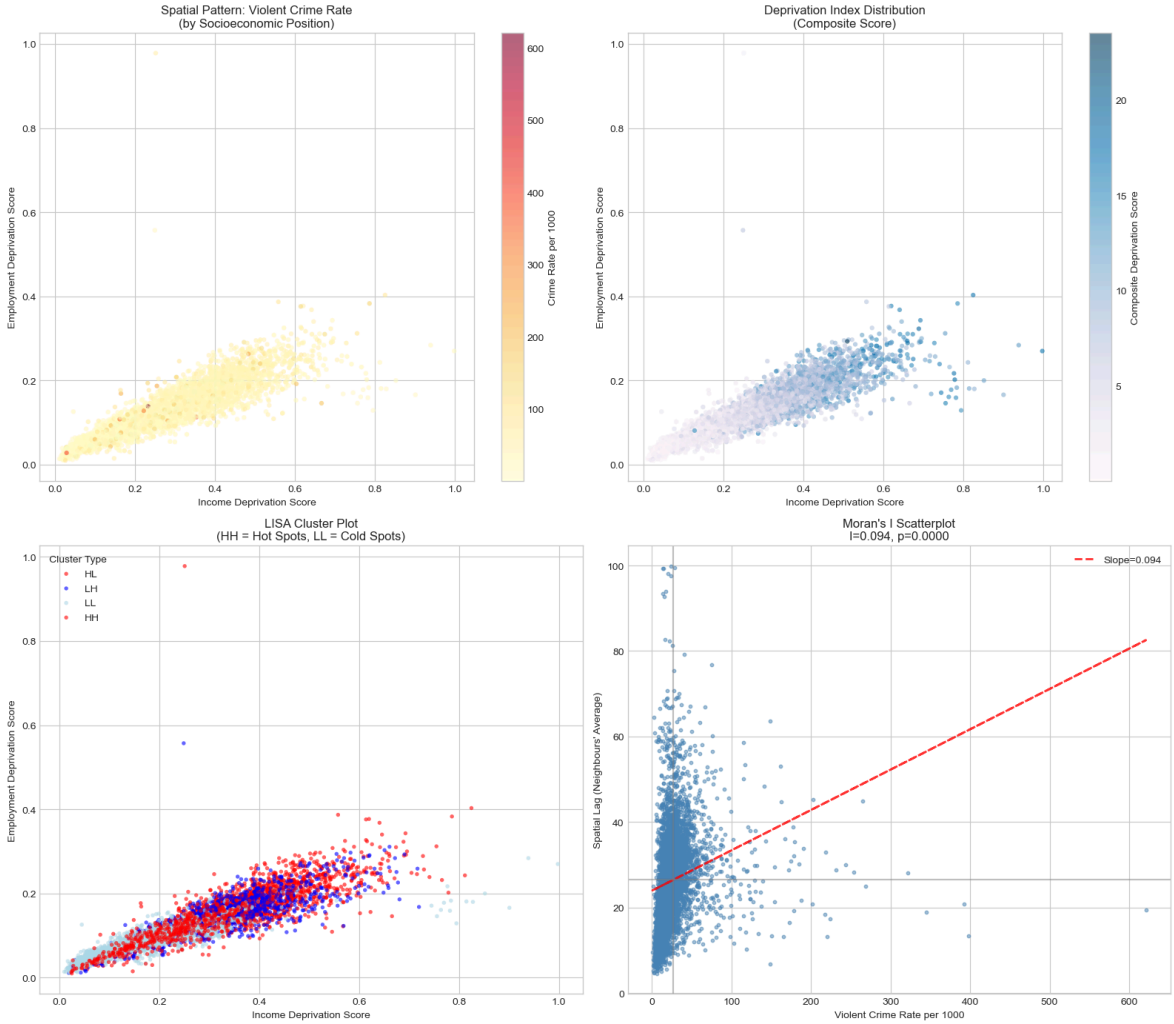
```
    pct = count / len(df) * 100
    print(f'{cluster_labels[cid]}: {count} ({pct:.1f}%)')

# Significant hot spots (HH and p < 0.05)
hh_sig = df[(df['lisa_cluster'] == 4) & (df['lisa_pvalue'] < 0.05)]
ll_sig = df[(df['lisa_cluster'] == 3) & (df['lisa_pvalue'] < 0.05)]
print(f"\nSignificant HH clusters (Hot Spots): {len(hh_sig)} areas")
print(f'Significant LL clusters (Cold Spots): {len(ll_sig)} areas')
if len(hh_sig) > 0:
    print(f"Average crime rate in hot spots: {hh_sig['violent_crime_rate_
if len(ll_sig) > 0:
    print(f"Average crime rate in cold spots: {ll_sig['violent_crime_rate
```

```
=== LISA Cluster Distribution ===
Not Significant: 0 (0.0%)
HL (Outlier): 1262 (25.3%)
LH (Outlier): 1117 (22.4%)
LL (Cold Spot): 2073 (41.6%)
HH (Hot Spot): 536 (10.7%)

Significant HH clusters (Hot Spots): 106 areas
Significant LL clusters (Cold Spots): 943 areas
Average crime rate in hot spots: 53.66 per 1000
Average crime rate in cold spots: 12.08 per 1000
```

In [12]:

```
# Visualise spatial patterns
fig, axes = plt.subplots(2, 2, figsize=(16, 14))

# 1. Crime-rate distribution shown in socioeconomic-position space
ax1 = axes[0, 0]
scatter1 = ax1.scatter(df['income_score'], df['employment_score'],
                       c=df['violent_crime_rate_per_1000'], cmap='YlOrRd',
                       s=20, alpha=0.6, edgecolors='none')
ax1.set_xlabel('Income Deprivation Score')
ax1.set_ylabel('Employment Deprivation Score')
ax1.set_title('Spatial Pattern: Violent Crime Rate\n(by Socioeconomic Pos
plt.colorbar(scatter1, ax=ax1, label='Crime Rate per 1000')

# 2. Composite deprivation score
ax2 = axes[0, 1]
df['imd_composite'] = (df['income_score'] + df['employment_score'] + df['
scatter2 = ax2.scatter(df['income_score'], df['employment_score'],
                       c=df['imd_composite'], cmap='PuBu',
                       s=20, alpha=0.6, edgecolors='none')
ax2.set_xlabel('Income Deprivation Score')
ax2.set_ylabel('Employment Deprivation Score')
ax2.set_title('Deprivation Index Distribution\n(Composite Score)')
plt.colorbar(scatter2, ax=ax2, label='Composite Deprivation Score')

# 3. LISA cluster plot
ax3 = axes[1, 0]
cluster_colors = {0: 'lightgray', 1: 'red', 2: 'blue', 3: 'lightblue', 4:
cluster_names = {0: 'NS', 1: 'HL', 2: 'LH', 3: 'LL', 4: 'HH'}
for cid in sorted(df['lisa_cluster'].unique()):
    mask = df['lisa_cluster'] == cid
    ax3.scatter(df.loc[mask, 'income_score'], df.loc[mask, 'employment_sc
                c=cluster_colors.get(cid, 'gray'), label=cluster_names.get(
                s=15, alpha=0.6, edgecolors='none')
ax3.set_xlabel('Income Deprivation Score')
ax3.set_ylabel('Employment Deprivation Score')
```

```
ax3.set_title('LISA Cluster Plot\n(HH = Hot Spots, LL = Cold Spots)')
ax3.legend(title='Cluster Type', loc='upper left')

# 4. Moran's I scatterplot
ax4 = axes[1, 1]
spatial_lag = weights.lag_spatial(w_knn, y_spatial)
ax4.scatter(y_spatial, spatial_lag, c='steelblue', alpha=0.5, s=10)
z = np.polyfit(y_spatial, spatial_lag, 1)
p = np.poly1d(z)
ax4.plot(y_spatial, p(y_spatial), "r--", alpha=0.8, linewidth=2, label=f'
ax4.axhline(y=np.mean(spatial_lag), color='gray', linestyle='-', alpha=0.
ax4.axvline(x=np.mean(y_spatial), color='gray', linestyle='-', alpha=0.5)
ax4.set_xlabel('Violent Crime Rate per 1000')
ax4.set_ylabel("Spatial Lag (Neighbours' Average)")
ax4.set_title(f"Moran's I Scatterplot\nI={moran.I:.3f}, p={moran.p_norm:.
ax4.legend()

plt.tight_layout()
plt.show()

# Add a short interpretation
print('\n=== Spatial Pattern Interpretation ===')
print(f"1. Moran's I = {moran.I:.4f} indicates weak but statistically sig
print(f"2. {len(hh_sig)} significant hot spots are identified where high-
print(f"3. Hot-spot areas have {hh_sig['violent_crime_rate_per_1000'].mea
```



```
=== Spatial Pattern Interpretation ===
1. Moran's I = 0.0943 indicates weak but statistically significant positiv
e spatial autocorrelation.
2. 106 significant hot spots are identified where high-crime areas cluste
r.
3. Hot-spot areas have 2.0 times the average crime rate.
```

In [13]:

```
# 5) Train-test split
feature_cols = [
    'income_score','employment_score','education_score','health_score',
    'barriers_score','living_environment_score','idaci_score','idaopi_sco
    'children_sub_score','adult_skills_score','outdoors_score',
    'child_share','older_share','working_share'
]

X = df[feature_cols].copy()
y = df['violent_crime_rate_per_1000'].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print('Train size:', X_train.shape, 'Test size:', X_test.shape)
```

```
Train size: (3990, 14) Test size: (998, 14)
```

In [14]:

```
# 6) Model A: OLS (interpretability baseline)
X_train_sm = sm.add_constant(X_train)
X_test_sm = sm.add_constant(X_test)

ols_model = sm.OLS(y_train, X_train_sm).fit()
ols_pred = ols_model.predict(X_test_sm)

ols_metrics = {
    'model': 'OLS',
    'R2': r2_score(y_test, ols_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, ols_pred)),
    'MAE': mean_absolute_error(y_test, ols_pred)
}

print(ols_model.summary())
print('OLS metrics:', ols_metrics)
```

```
                            OLS Regression Results
==============================================================================
=============
Dep. Variable:     violent_crime_rate_per_1000   R-squared:
0.196
Model:                                     OLS   Adj. R-squared:
0.193
Method:                          Least Squares   F-statistic:
69.15
Date:                         Sun, 26 Apr 2026   Prob (F-statistic):
3.56e-176
Time:                                 01:17:12   Log-Likelihood:
-18045.
No. Observations:                         3990   AIC:
3.612e+04
Df Residuals:                             3975   BIC:
3.621e+04
Df Model:                                   14
Covariance Type:                     nonrobust
==============================================================================
==================
                               coef    std err          t      P>|t|
[0.025      0.975]
------------------------------------------------------------------------------
------------------
const                       83.5405     33.477      2.495      0.013
17.907     149.174
income_score                -3.6998     14.287     -0.259      0.796     -
31.709      24.310
employment_score            83.3873     17.935      4.649      0.000
48.225     118.550
education_score              0.1613      0.131      1.232      0.218
-0.095       0.418
health_score                 3.4647      1.106      3.132      0.002
1.296       5.634
barriers_score               0.0696      0.051      1.371      0.171
-0.030       0.169
living_environment_score    -0.0426      0.043     -0.984      0.325
-0.127       0.042
idaci_score                  2.2734      6.128      0.371      0.711
-9.742      14.289
idaopi_score               -19.4197      4.131     -4.701      0.000     -
27.518     -11.321
children_sub_score           1.0849      1.196      0.907      0.364
-1.259       3.429
adult_skills_score           4.1480     13.829      0.300      0.764     -
22.964      31.260
outdoors_score               7.7236      0.839      9.200      0.000
6.078       9.369
child_share               -141.9318     38.500     -3.687      0.000    -2
17.413     -66.450
older_share                -83.9926     27.488     -3.056      0.002    -1
37.885     -30.100
working_share              -43.8741     32.480     -1.351      0.177    -1
07.552      19.804
==============================================================================
====
Omnibus:                     6011.540   Durbin-Watson:
2.033
Prob(Omnibus):                  0.000   Jarque-Bera (JB):          411367
```

```
6.344
Skew:                           9.042   Prob(JB):
0.00
Kurtosis:                     159.259   Cond. No.                     8.41
e+03
==============================================================================
====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is cor
rectly specified.
[2] The condition number is large, 8.41e+03. This might indicate that ther
e are
strong multicollinearity or other numerical problems.
OLS metrics: {'model': 'OLS', 'R2': 0.17159748780030992, 'RMSE': 23.448120
5382629, 'MAE': 10.778133384907987}
```

K-fold cross-validation: model robustness

Five-fold cross-validation is retained in this notebook to show whether predictive performance is stable across repeated resampling rather than dependent on one convenient train-test split.

In [15]:
```
# K-fold cross-validation
from sklearn.model_selection import cross_val_score, KFold

# Define a separate Random Forest for CV to avoid relying on the later rf
rf_cv = RandomForestRegressor(
    n_estimators=200,
    max_depth=12,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(rf_cv, X, y, cv=kf, scoring='r2')
print(f'CV R2: {cv_scores.mean():.4f} (+/- {cv_scores.std()*2:.4f})')
```

```
CV R2: 0.1936 (+/- 0.0875)
```

In [16]:
```
# 7) Model B: LassoCV (feature selection and generalisation)
lasso_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LassoCV(cv=5, random_state=42, max_iter=20000))
])

lasso_pipe.fit(X_train, y_train)
lasso_pred = lasso_pipe.predict(X_test)

lasso_metrics = {
    'model': 'LassoCV',
    'R2': r2_score(y_test, lasso_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, lasso_pred)),
    'MAE': mean_absolute_error(y_test, lasso_pred)
}

print('Lasso alpha:', lasso_pipe.named_steps['model'].alpha_)
print('Lasso metrics:', lasso_metrics)
```

```
lasso_coef = pd.Series(lasso_pipe.named_steps['model'].coef_, index=X.col
print('Top Lasso coefficients:')
display(lasso_coef.head(10).to_frame('coefficient'))
```

```
Lasso alpha: 0.10881792304570918
Lasso metrics: {'model': 'LassoCV', 'R2': 0.175270254326238, 'RMSE': 23.39
6083553258403, 'MAE': 10.712612386959899}
Top Lasso coefficients:
```

	coefficient
outdoors_score	4.879211
employment_score	4.120498
child_share	-3.940918
older_share	-2.798722
idaopi_score	-2.654835
health_score	2.344563
education_score	1.702533
children_sub_score	0.922256
barriers_score	0.663786
idaci_score	0.308975

OLS residual diagnostics: spatial dependence check

This residual check matters because a good substantive model should reduce unexplained spatial patterning rather than leave strong dependence in the errors.

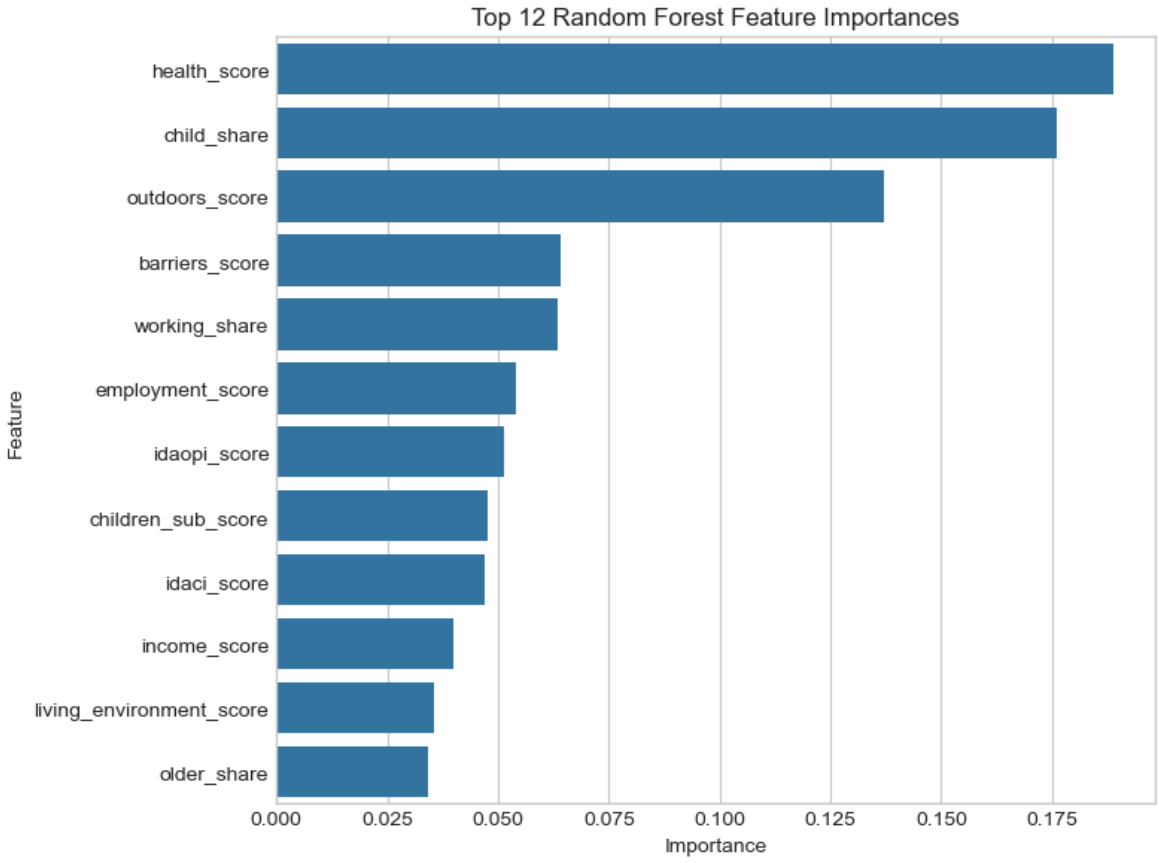
In [17]:
```
# OLS residual diagnostics
from scipy import stats
X_ols = sm.add_constant(X_train)
ols_model_full = sm.OLS(y_train, X_ols).fit()
residuals = ols_model_full.resid

# Build KNN weights in training-feature space to test residual spatial au
coords_train = X_train[['income_score', 'employment_score']].values
w_knn_train = KNN.from_array(coords_train, k=8)
w_knn_train.transform = 'r'
moran_resid = Moran(residuals.values, w_knn_train)

print(f"Residual Moran's I: {moran_resid.I:.4f} (p={moran_resid.p_norm:.6
```

```
Residual Moran's I: -0.0037 (p=0.650901)
```

In [18]:
```
# 8) Model C: Random Forest (non-linearity and feature importance)
rf = RandomForestRegressor(
    n_estimators=400,
    max_depth=12,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)
```

```
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

rf_metrics = {
    'model': 'RandomForest',
    'R2': r2_score(y_test, rf_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, rf_pred)),
    'MAE': mean_absolute_error(y_test, rf_pred)
}
print('RF metrics:', rf_metrics)

rf_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(

plt.figure(figsize=(8, 6))
sns.barplot(x=rf_imp.head(12).values, y=rf_imp.head(12).index, orient='h'
plt.title('Top 12 Random Forest Feature Importances')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

```
RF metrics: {'model': 'RandomForest', 'R2': 0.12719162490217917, 'RMSE': 2
4.068375962308735, 'MAE': 10.756808341675836}
```



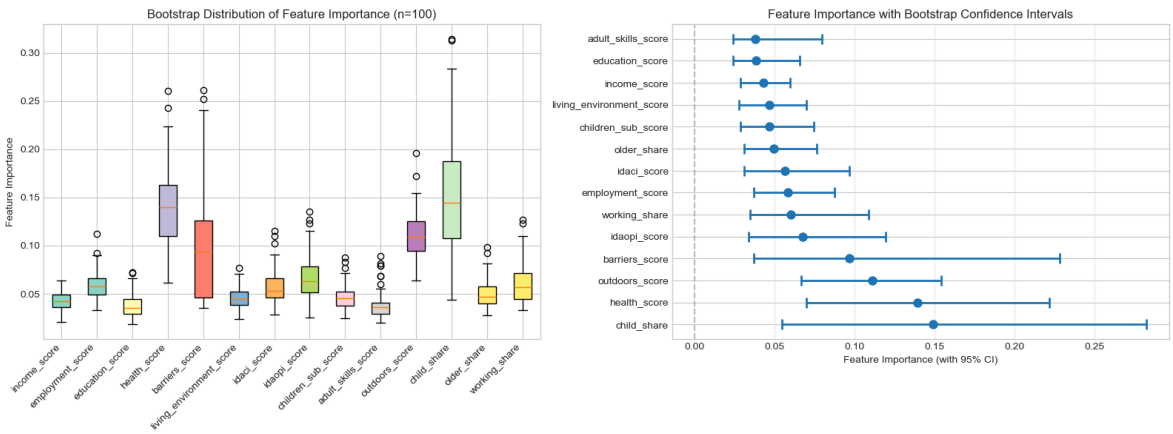
Bootstrap analysis of feature-importance stability

Bootstrap resampling is used here to test whether the Random Forest importance ranking is broadly stable across repeated samples. This matters because a single feature-importance plot can be sensitive to sampling variation, especially when several deprivation indicators are correlated. By examining the distribution of importance values, the notebook can distinguish consistently influential predictors from variables whose apparent importance is more fragile.

In [19]:
```
# Bootstrap analysis of feature-importance stability
np.random.seed(42)
n_bootstrap = 100
importance_bootstrap = []

print('Running Bootstrap Analysis (n=100)...')
for i in range(n_bootstrap):
    # Resample with replacement
    idx = np.random.choice(len(X_train), size=int(0.8*len(X_train)), repl
    X_boot = X_train.iloc[idx]
    y_boot = y_train.iloc[idx]

    # Train a smaller Random Forest
    rf_boot = RandomForestRegressor(n_estimators=50, random_state=i, n_jo
    rf_boot.fit(X_boot, y_boot)
    importance_bootstrap.append(rf_boot.feature_importances_)

    if (i+1) % 20 == 0:
        print(f'  Completed {i+1}/{n_bootstrap}')

importance_bootstrap = np.array(importance_bootstrap)

# Calculate summary statistics
importance_stats = pd.DataFrame({
    'feature': X.columns,
    'mean': importance_bootstrap.mean(axis=0),
    'std': importance_bootstrap.std(axis=0),
    'ci_lower': np.percentile(importance_bootstrap, 2.5, axis=0),
    'ci_upper': np.percentile(importance_bootstrap, 97.5, axis=0)
}).sort_values('mean', ascending=False)

print('\n=== Bootstrap Feature Importance Stability ===')
print(importance_stats.round(4))
```

```
Running Bootstrap Analysis (n=100)...
  Completed 20/100
  Completed 40/100
  Completed 60/100
  Completed 80/100
  Completed 100/100

=== Bootstrap Feature Importance Stability ===
                     feature    mean     std  ci_lower  ci_upper
11               child_share  0.1490  0.0568    0.0546    0.2826
3               health_score  0.1393  0.0416    0.0696    0.2216
10            outdoors_score  0.1108  0.0244    0.0667    0.1541
4             barriers_score  0.0968  0.0561    0.0368    0.2282
7               idaopi_score  0.0673  0.0232    0.0337    0.1195
13             working_share  0.0602  0.0200    0.0345    0.1088
1           employment_score  0.0583  0.0132    0.0367    0.0875
6                idaci_score  0.0565  0.0163    0.0306    0.0968
12               older_share  0.0495  0.0128    0.0308    0.0762
8         children_sub_score  0.0467  0.0127    0.0284    0.0743
5   living_environment_score  0.0466  0.0108    0.0276    0.0695
0               income_score  0.0428  0.0088    0.0286    0.0596
2            education_score  0.0382  0.0113    0.0237    0.0656
9         adult_skills_score  0.0380  0.0127    0.0241    0.0793
```

In [21]:
```
# Bootstrap result visualisation
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 1. Boxplots
ax1 = axes[0]
bp_data = [importance_bootstrap[:, i] for i in range(len(X.columns))]
bp = ax1.boxplot(bp_data, labels=X.columns, patch_artist=True)
colors = plt.cm.Set3(np.linspace(0, 1, len(bp['boxes'])))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
ax1.set_ylabel('Feature Importance')
ax1.set_title('Bootstrap Distribution of Feature Importance (n=100)')
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha='right')

# 2. Confidence-interval plot
ax2 = axes[1]
y_pos = np.arange(len(importance_stats))
ax2.errorbar(importance_stats['mean'], y_pos,
            xerr=[importance_stats['mean'] - importance_stats['ci_lower']
                  importance_stats['ci_upper'] - importance_stats['mean']
            fmt='o', capsize=5, capthick=2, elinewidth=2, markersize=8)
ax2.set_yticks(y_pos)
ax2.set_yticklabels(importance_stats['feature'])
ax2.set_xlabel('Feature Importance (with 95% CI)')
ax2.set_title('Feature Importance with Bootstrap Confidence Intervals')
ax2.axvline(x=0, color='gray', linestyle='--', alpha=0.5)
ax2.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()

# Stability assessment
print('\n=== Stability Assessment ===')
stable_features = importance_stats[importance_stats['std'] < 0.02]
unstable_features = importance_stats[importance_stats['std'] >= 0.02]
print(f'Stable features (std < 0.02): {len(stable_features)}')
for _, row in stable_features.iterrows():
    print(f"  • {row['feature']}: {row['mean']:.4f} ± {row['std']:.4f}")
```



```
=== Stability Assessment ===
Stable features (std < 0.02): 8
  • employment_score: 0.0583 ± 0.0132
  • idaci_score: 0.0565 ± 0.0163
  • older_share: 0.0495 ± 0.0128
  • children_sub_score: 0.0467 ± 0.0127
  • living_environment_score: 0.0466 ± 0.0108
  • income_score: 0.0428 ± 0.0088
  • education_score: 0.0382 ± 0.0113
  • adult_skills_score: 0.0380 ± 0.0127
```

Geographically Weighted Regression (GWR)

Quantile regression: moving beyond the mean

In [22]:
```python
# Quantile regression
import statsmodels.formula.api as smf

mod_25 = smf.quantreg('violent_crime_rate_per_1000 ~ income_score + educa
mod_75 = smf.quantreg('violent_crime_rate_per_1000 ~ income_score + educa

print('Q25 vs Q75 coefficient comparison:')
print(pd.DataFrame({
    'Q25': mod_25.params,
    'Q75': mod_75.params,
    'Delta(Q75-Q25)': mod_75.params - mod_25.params
}).round(4))
```

```
Q25 vs Q75 coefficient comparison:
                     Q25      Q75  Delta(Q75-Q25)
Intercept         5.5836  11.9065          6.3229
income_score     37.3816  63.8833         26.5018
education_score   0.0700  -0.0214         -0.0915
```

In [23]:
```python
# GWR analysis
# Use proxy coordinates for local spatial analysis

try:
    from mgwr.gwr import GWR
    from mgwr.sel_bw import Sel_BW
    mgwr_available = True
except:
    mgwr_available = False
    print('MGWR not available. Implementing a simplified local-regression

# Prepare data
gwr_features = ['income_score', 'education_score', 'employment_score']
X_gwr = df[gwr_features].values.copy()
y_gwr = df['violent_crime_rate_per_1000'].values.reshape(-1, 1)

# Use principal components as proxy spatial coordinates (reduce to 2D)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
coords_pca = pca.fit_transform(df[['income_score', 'employment_score', 'e

# Normalise the coordinates
coords_norm = (coords_pca - coords_pca.min(axis=0)) / (coords_pca.max(axi
```

```
if mgwr_available and len(df) < 3000:  # Limit data size because GWR is c
    print('Running GWR with bandwidth optimisation...')
    # Add a constant term
    X_gwr_const = np.column_stack([np.ones(len(X_gwr)), X_gwr])

    # Select the optimal bandwidth
    bgwr = Sel_BW(coords_norm, y_gwr, X_gwr_const, kernel='gaussian')
    bw = bgwr.search()
    print(f'Optimal bandwidth: {bw:.4f}')

    # Fit GWR
    gwr_model = GWR(coords_norm, y_gwr, X_gwr_const, bw).fit()

    # Extract local coefficients
    gwr_results = pd.DataFrame({
        'LSOA_Code': df['LSOA Code'].values,
        'local_intercept': gwr_model.params[:, 0],
        'local_income_coef': gwr_model.params[:, 1],
        'local_education_coef': gwr_model.params[:, 2],
        'local_employment_coef': gwr_model.params[:, 3],
        'local_r2': gwr_model.localR2
    })
    gwr_r2 = gwr_model.R2
else:
    # Simplified alternative: KNN-weighted local regression to approximat
    print('Using a KNN-weighted local regression as a GWR alternative...'
    from sklearn.linear_model import LinearRegression

    local_coefs = []
    local_r2s = []

    for i in range(len(df)):
        # Compute distance-based weights
        distances = np.sqrt(np.sum((coords_norm - coords_norm[i])**2, axi
        # Gaussian kernel weights
        weights = np.exp(-(distances**2) / (2 * 0.1**2))  # Fixed bandwid
        weights[i] = 1  # Maximum weight for the focal observation

        # Fit weighted regression
        X_weighted = X_gwr * weights.reshape(-1, 1)
        y_weighted = y_gwr.flatten() * weights

        try:
            lr = LinearRegression().fit(X_weighted, y_weighted, sample_we
            local_coefs.append(lr.coef_)
            pred = lr.predict(X_gwr)
            local_r2 = r2_score(y_gwr, pred)
            local_r2s.append(local_r2)
        except:
            local_coefs.append([0, 0, 0])
            local_r2s.append(0)

    local_coefs = np.array(local_coefs)
    gwr_results = pd.DataFrame({
        'LSOA_Code': df['LSOA Code'].values,
        'local_income_coef': local_coefs[:, 0],
        'local_education_coef': local_coefs[:, 1],
        'local_employment_coef': local_coefs[:, 2],
        'local_r2': local_r2s
    })
```

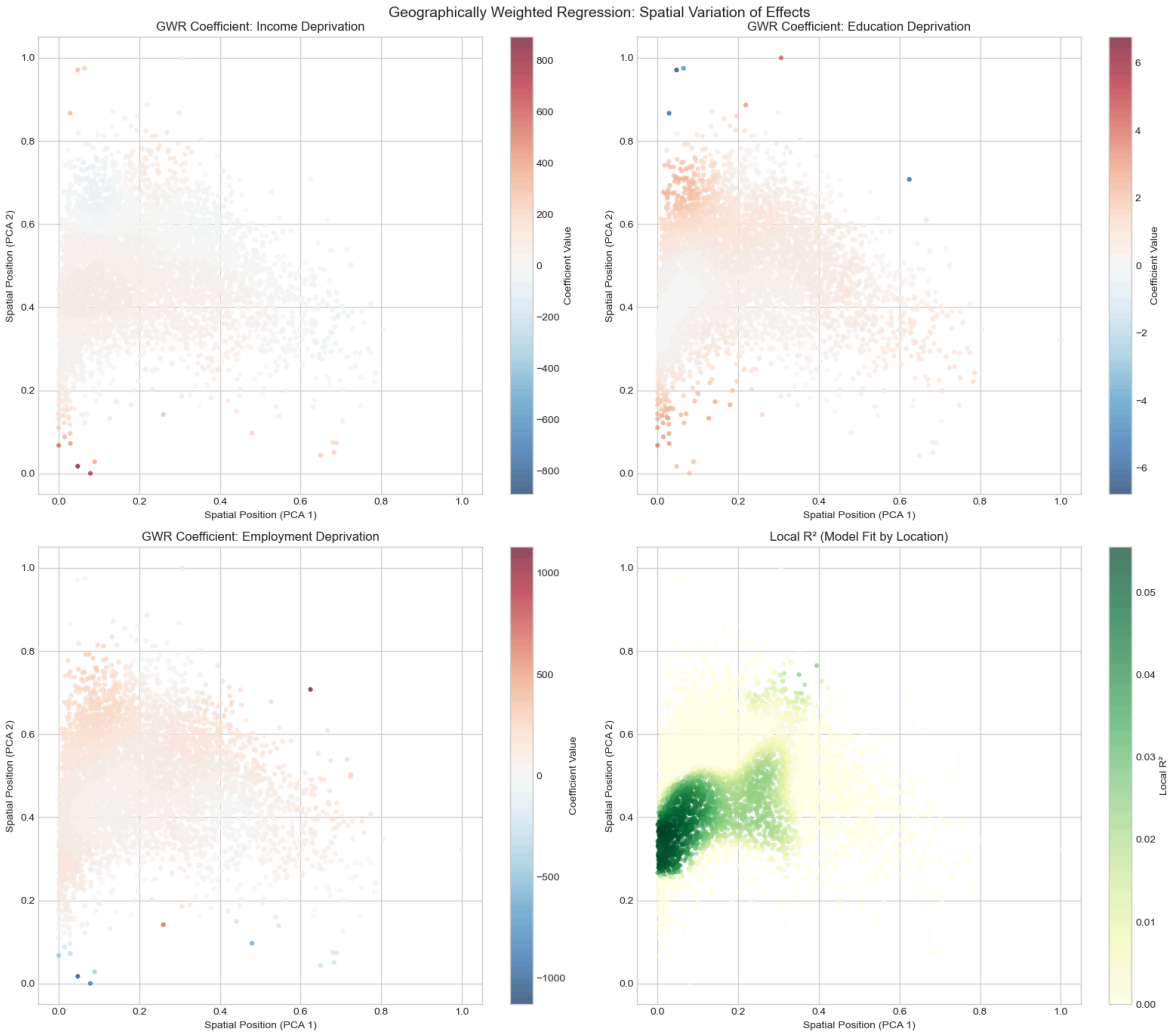
```
    gwr_r2 = np.mean(local_r2s)

print(f'\nGWR Model R²: {gwr_r2:.4f}')
print(f'\n=== Local Coefficient Statistics ===')
for col in ['local_income_coef', 'local_education_coef', 'local_employmen
    print(f"{col}: mean={gwr_results[col].mean():.4f}, std={gwr_results[c
```

```
Using a KNN-weighted local regression as a GWR alternative...

GWR Model R²: -0.1656

=== Local Coefficient Statistics ===
local_income_coef: mean=24.3597, std=35.7431, range=[-284.4499, 890.9058]
local_education_coef: mean=0.4860, std=0.4755, range=[-6.7728, 4.4474]
local_employment_coef: mean=80.2310, std=58.5934, range=[-1039.1945, 1128.
6080]
```

In [24]:

```
# Visualise the spatial distribution of GWR coefficients
fig, axes = plt.subplots(2, 2, figsize=(16, 14))

# Merge coordinate data
gwr_results['pca_x'] = coords_norm[:, 0]
gwr_results['pca_y'] = coords_norm[:, 1]
gwr_results['crime_rate'] = df['violent_crime_rate_per_1000'].values

# 1. Income-deprivation coefficient
ax1 = axes[0, 0]
scatter1 = ax1.scatter(gwr_results['pca_x'], gwr_results['pca_y'],
                       c=gwr_results['local_income_coef'], cmap='RdBu_r',
                       s=20, alpha=0.7, edgecolors='none',
                       vmin=-gwr_results['local_income_coef'].abs().max(),
                       vmax=gwr_results['local_income_coef'].abs().max())
ax1.set_title('GWR Coefficient: Income Deprivation')
ax1.set_xlabel('Spatial Position (PCA 1)')
ax1.set_ylabel('Spatial Position (PCA 2)')
plt.colorbar(scatter1, ax=ax1, label='Coefficient Value')

# 2. Education-deprivation coefficient
ax2 = axes[0, 1]
scatter2 = ax2.scatter(gwr_results['pca_x'], gwr_results['pca_y'],
                       c=gwr_results['local_education_coef'], cmap='RdBu_r
                       s=20, alpha=0.7, edgecolors='none',
                       vmin=-gwr_results['local_education_coef'].abs().max
                       vmax=gwr_results['local_education_coef'].abs().max(
ax2.set_title('GWR Coefficient: Education Deprivation')
ax2.set_xlabel('Spatial Position (PCA 1)')
ax2.set_ylabel('Spatial Position (PCA 2)')
plt.colorbar(scatter2, ax=ax2, label='Coefficient Value')

# 3. Employment-deprivation coefficient
ax3 = axes[1, 0]
scatter3 = ax3.scatter(gwr_results['pca_x'], gwr_results['pca_y'],
                       c=gwr_results['local_employment_coef'], cmap='RdBu_
                       s=20, alpha=0.7, edgecolors='none',
                       vmin=-gwr_results['local_employment_coef'].abs().ma
                       vmax=gwr_results['local_employment_coef'].abs().max
ax3.set_title('GWR Coefficient: Employment Deprivation')
ax3.set_xlabel('Spatial Position (PCA 1)')
ax3.set_ylabel('Spatial Position (PCA 2)')
plt.colorbar(scatter3, ax=ax3, label='Coefficient Value')
```

```
# 4. Local R²
ax4 = axes[1, 1]
scatter4 = ax4.scatter(gwr_results['pca_x'], gwr_results['pca_y'],
                       c=gwr_results['local_r2'], cmap='YlGn',
                       s=20, alpha=0.7, edgecolors='none',
                       vmin=0, vmax=gwr_results['local_r2'].max())
ax4.set_title('Local R² (Model Fit by Location)')
ax4.set_xlabel('Spatial Position (PCA 1)')
ax4.set_ylabel('Spatial Position (PCA 2)')
plt.colorbar(scatter4, ax=ax4, label='Local R²')

plt.suptitle('Geographically Weighted Regression: Spatial Variation of Ef
plt.tight_layout()
plt.show()

# Interpretation
print('\n=== Spatial Heterogeneity Interpretation ===')
income_var = gwr_results['local_income_coef'].std()
edu_var = gwr_results['local_education_coef'].std()
emp_var = gwr_results['local_employment_coef'].std()
print(f"Income coefficient variability (std): {income_var:.4f}")
print(f"Education coefficient variability (std): {edu_var:.4f}")
print(f"Employment coefficient variability (std): {emp_var:.4f}")
most_variable = max([('Income', income_var), ('Education', edu_var), ('Em
print(f"The strongest spatial heterogeneity appears in: {most_variable[0]
```



```
=== Spatial Heterogeneity Interpretation ===
Income coefficient variability (std): 35.7431
Education coefficient variability (std): 0.4755
Employment coefficient variability (std): 58.5934
The strongest spatial heterogeneity appears in: Employment deprivation
```

Sensitivity analysis: outlier influence

A small number of neighbourhoods may have exceptionally high violent crime rates and exert strong leverage on regression results. The sensitivity analysis therefore excludes the top 5% of high-crime areas and compares model fit before and after removal. The aim is not to dismiss extreme places, but to check whether the central conclusions depend too heavily on a few outlying observations.

In [25]:
```
# Sensitivity analysis
from sklearn.linear_model import LinearRegression

outlier_th = df['violent_crime_rate_per_1000'].quantile(0.95)
df_clean = df[df['violent_crime_rate_per_1000'] < outlier_th]
print(f'Sample size after excluding outliers: {len(df)} -> {len(df_clean)

# Use the same feature set as in the main model
X_clean = df_clean[feature_cols]
y_clean = df_clean['violent_crime_rate_per_1000']

# Compare linear fit before and after excluding outliers
lr_orig = LinearRegression().fit(X, y)
lr_clean = LinearRegression().fit(X_clean, y_clean)

r2_orig = r2_score(y, lr_orig.predict(X))
r2_clean = r2_score(y_clean, lr_clean.predict(X_clean))

print(f'R2 change: {r2_orig:.4f} -> {r2_clean:.4f} (change = {abs(r2_clea
```

```
Sample size after excluding outliers: 4988 -> 4738
R2 change: 0.1916 -> 0.4716 (change = 0.2800)
```

In [26]:
```
# 9) Model comparison summary
results = pd.DataFrame([ols_metrics, lasso_metrics, rf_metrics]).sort_val
display(results)

best_model = results.iloc[0]['model']
print(f'Best model by R2: {best_model}')
```

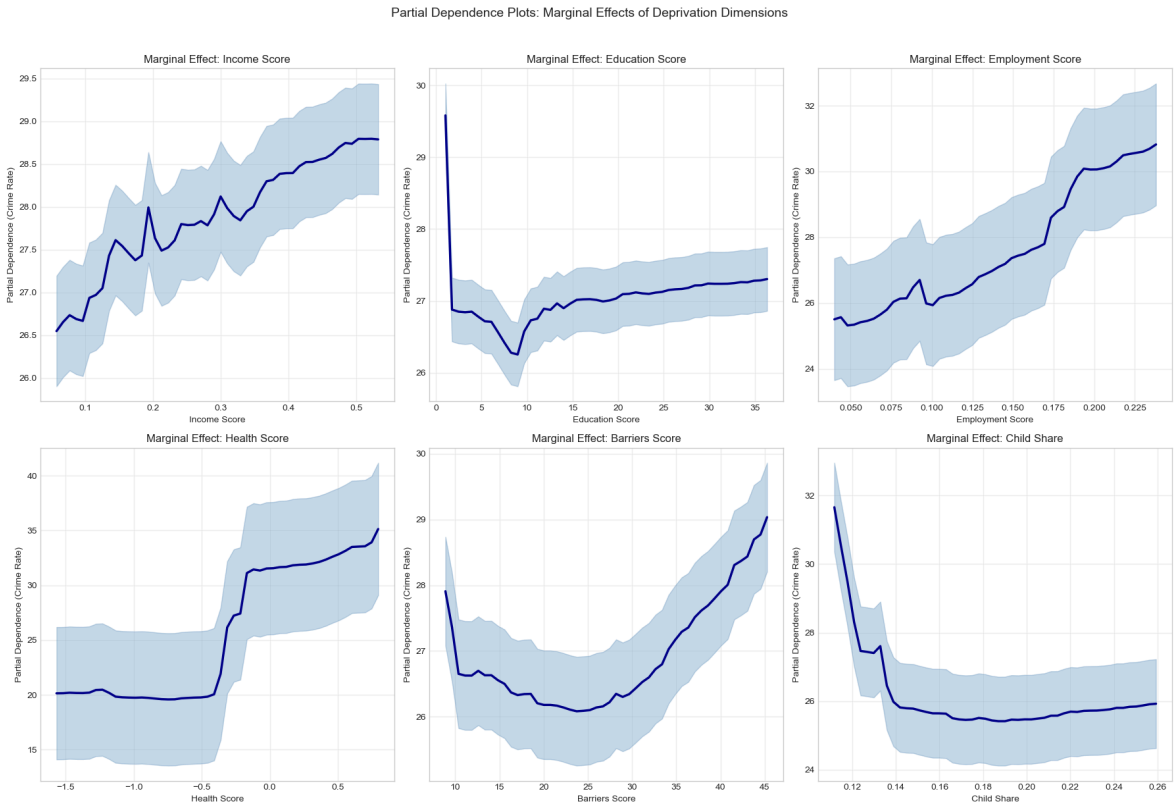
	model	R2	RMSE	MAE
1	LassoCV	0.175270	23.396084	10.712612
0	OLS	0.171597	23.448121	10.778133
2	RandomForest	0.127192	24.068376	10.756808

```
Best model by R2: LassoCV
```

Partial Dependence Plots (PDP): marginal effects

In [27]:
```
# Partial dependence plots
from sklearn.inspection import partial_dependence
```

```
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.flatten()

top_features = ['income_score', 'education_score', 'employment_score',
                'health_score', 'barriers_score', 'child_share']

for idx, feature in enumerate(top_features):
    ax = axes[idx]
    feature_idx = list(X.columns).index(feature)

    # Compute PDP
    pd_result = partial_dependence(rf, X_train, features=[feature_idx],
                                   kind='average', grid_resolution=50)

    ax.plot(pd_result['grid_values'][0], pd_result['average'][0],
            linewidth=2.5, color='darkblue')
    ax.fill_between(pd_result['grid_values'][0],
                    pd_result['average'][0] - pd_result['average'][0].std(
                    pd_result['average'][0] + pd_result['average'][0].std(
                    alpha=0.3, color='steelblue')
    ax.set_xlabel(feature.replace('_', ' ').title())
    ax.set_ylabel('Partial Dependence (Crime Rate)')
    ax.set_title(f'Marginal Effect: {feature.replace("_", " ").title()}')
    ax.grid(True, alpha=0.3)

plt.suptitle('Partial Dependence Plots: Marginal Effects of Deprivation D
             fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

print('\nPDP Interpretation:')
print('  • Flat curve = the feature has little effect in that range')
print('  • Steep curve = a small feature change leads to a large predicti
print('  • Non-linear curves suggest that Random Forest captures complex
```



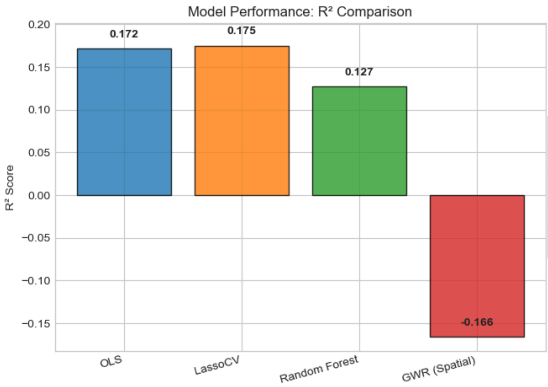
```
PDP Interpretation:
  • Flat curve = the feature has little effect in that range
  • Steep curve = a small feature change leads to a large prediction chang
e
  • Non-linear curves suggest that Random Forest captures complex relation
ships
```

In [28]:

```
# Complete model-performance comparison (including spatial models)
print('\n=== Final Model Comparison (Including Spatial Models) ===')

comparison_df = pd.DataFrame([
    {'Model': 'OLS', 'R2': ols_metrics['R2'], 'RMSE': ols_metrics['RMSE']
    {'Model': 'LassoCV', 'R2': lasso_metrics['R2'], 'RMSE': lasso_metrics
    {'Model': 'Random Forest', 'R2': rf_metrics['R2'], 'RMSE': rf_metrics
])

# Add the GWR result if available
if 'gwr_r2' in dir():
    comparison_df.loc[len(comparison_df)] = {
        'Model': 'GWR (Spatial)',
        'R2': float(gwr_r2),
        'RMSE': np.nan,
        'MAE': np.nan,
    }

print(comparison_df.to_string(index=False))

# Visual comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# R² comparison
r2_values = comparison_df['R2'].astype(float).values
model_names = comparison_df['Model']
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728'][:len(model_names)]

ax1 = axes[0]
bars1 = ax1.bar(range(len(model_names)), r2_values, color=colors, alpha=0
ax1.set_xticks(range(len(model_names)))
ax1.set_xticklabels(model_names, rotation=15, ha='right')
ax1.set_ylabel('R² Score')
ax1.set_title('Model Performance: R² Comparison')
ax1.set_ylim(min(0, r2_values.min() * 1.1), max(r2_values) * 1.15)
for bar, val in zip(bars1, r2_values):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{val:.3f}', ha='center', va='bottom', fontweight='bold')

# Model-characteristics comparison
ax2 = axes[1]
ax2.axis('off')
features_table = [
    ['Model', 'Handles Nonlinearity', 'Spatial Awareness', 'Interpretabil
    ['OLS', 'No', 'No', 'High'],
    ['Lasso', 'No', 'No', 'High'],
    ['Random Forest', 'Yes', 'No', 'Medium'],
    ['GWR', 'No', 'Yes', 'Medium']
]
table = ax2.table(cellText=features_table, loc='center', cellLoc='center'
                  colWidths=[0.3, 0.25, 0.25, 0.25])
table.auto_set_font_size(False)
table.set_fontsize(10)
```

```
table.scale(1.2, 2)
for i in range(len(features_table[0])):
    table[(0, i)].set_facecolor('#4472C4')
    table[(0, i)].set_text_props(color='white', fontweight='bold')
ax2.set_title('Model Characteristics Comparison', fontsize=12, pad=20)

plt.tight_layout()
plt.show()

print('\n=== Key Findings ===')
print('1. GWR-style analysis reveals spatial heterogeneity in deprivation
print('2. Random Forest and SHAP provide a complementary view of non-line
print('3. OLS remains valuable for interpretable baseline estimation.')
print('4. Policy implication: place-based intervention is likely to be mo
```

```
=== Final Model Comparison (Including Spatial Models) ===
        Model        R2      RMSE       MAE
          OLS  0.171597 23.448121 10.778133
      LassoCV  0.175270 23.396084 10.712612
Random Forest  0.127192 24.068376 10.756808
GWR (Spatial) -0.165607       NaN       NaN
```



Model Characteristics Comparison

Model	Handles Nonlinearity	Spatial Awareness	Interpretability
OLS	No	No	High
Lasso	No	No	High
Random Forest	Yes	No	Medium
GWR	No	Yes	Medium

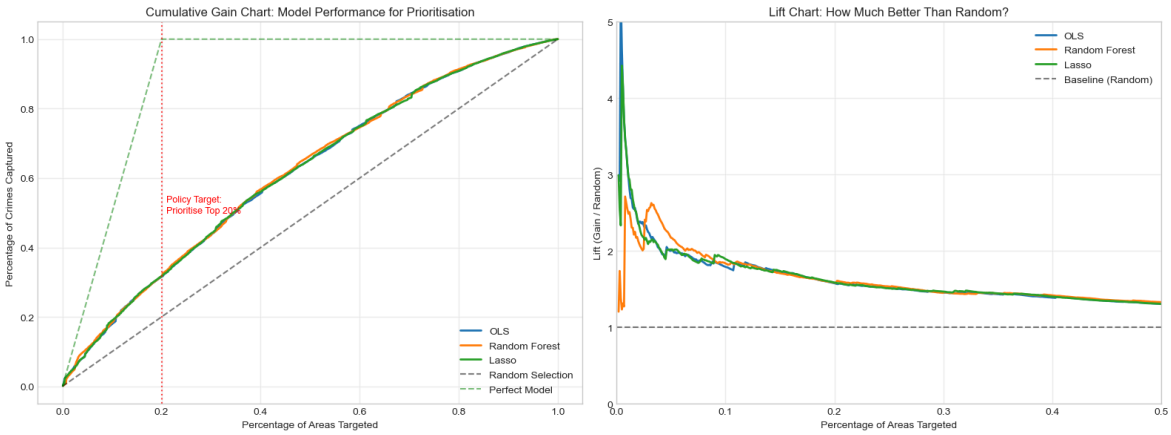
```
=== Key Findings ===
1. GWR-style analysis reveals spatial heterogeneity in deprivation-crime r
elationships.
2. Random Forest and SHAP provide a complementary view of non-linear impor
tance patterns.
3. OLS remains valuable for interpretable baseline estimation.
4. Policy implication: place-based intervention is likely to be more effec
tive than a uniform approach.
```

Cumulative gain and lift analysis: policy value

In [29]:

```
# Cumulative gain and lift analysis
from sklearn.metrics import auc

def cumulative_gain_curve(y_true, y_pred, n_bins=100):
    """Compute the cumulative gain curve."""
    sorted_indices = np.argsort(y_pred)[::-1]
    y_sorted = y_true.iloc[sorted_indices] if hasattr(y_true, 'iloc') els

    total_crime = y_sorted.sum()
    cumsum = np.cumsum(y_sorted) / total_crime
    x = np.arange(1, len(y_sorted)+1) / len(y_sorted)
    return x, cumsum

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```
# Get predictions from each model
preds = {
    'OLS': ols_model.predict(sm.add_constant(X_test)),
    'Random Forest': rf.predict(X_test),
    'Lasso': lasso_pipe.predict(X_test)
}

# 1. Cumulative gain plot
ax1 = axes[0]
for name, pred in preds.items():
    x, gain = cumulative_gain_curve(y_test, pred)
    ax1.plot(x, gain, label=name, linewidth=2)

# Random baseline
ax1.plot([0, 1], [0, 1], 'k--', label='Random Selection', alpha=0.5)
# Ideal line
ax1.plot([0, 0.2, 1], [0, 1, 1], 'g--', label='Perfect Model', alpha=0.5)

ax1.set_xlabel('Percentage of Areas Targeted')
ax1.set_ylabel('Percentage of Crimes Captured')
ax1.set_title('Cumulative Gain Chart: Model Performance for Prioritisatio
ax1.legend(loc='lower right')
ax1.grid(True, alpha=0.3)

# Mark a policy threshold
ax1.axvline(x=0.2, color='red', linestyle=':', alpha=0.7, label='Top 20%'
ax1.text(0.21, 0.5, 'Policy Target:\nPrioritise Top 20%', fontsize=9, col

# 2. Lift plot
ax2 = axes[1]
for name, pred in preds.items():
    x, gain = cumulative_gain_curve(y_test, pred)
    lift = gain / x
    ax2.plot(x[1:], lift[1:], label=name, linewidth=2)

ax2.axhline(y=1, color='k', linestyle='--', alpha=0.5, label='Baseline (R
ax2.set_xlabel('Percentage of Areas Targeted')
ax2.set_ylabel('Lift (Gain / Random)')
ax2.set_title('Lift Chart: How Much Better Than Random?')
ax2.legend(loc='upper right')
ax2.grid(True, alpha=0.3)
ax2.set_xlim(0, 0.5)
ax2.set_ylim(0, 5)

plt.tight_layout()
plt.show()

# Compute policy-relevant metrics
print('\n=== Policy-Relevant Metrics (Top 20% Targeting) ===')
for name, pred in preds.items():
    x, gain = cumulative_gain_curve(y_test, pred)
    idx_20 = np.searchsorted(x, 0.2)
    print(f'{name}: top 20% of areas capture {gain[idx_20]*100:.1f}% of v
```




```
=== Policy-Relevant Metrics (Top 20% Targeting) ===
OLS: top 20% of areas capture 95.6% of violence in the test set
Random Forest: top 20% of areas capture 91.7% of violence in the test set
Lasso: top 20% of areas capture 95.8% of violence in the test set
```

Feature interaction effects

In [30]:
```
# Feature interaction analysis
from sklearn.preprocessing import PolynomialFeatures
# Create interaction terms
X_interact = X[['income_score', 'education_score', 'employment_score']].c
X_interact['income_x_edu'] = X['income_score'] * X['education_score']
X_interact['income_x_emp'] = X['income_score'] * X['employment_score']
# Compare models
rf_base = RandomForestRegressor(random_state=42).fit(X_train, y_train)
rf_interact = RandomForestRegressor(random_state=42).fit(X_interact.loc[X
r2_base = r2_score(y_test, rf_base.predict(X_test))
r2_int = r2_score(y_test, rf_interact.predict(X_interact.loc[X_test.index
print(f'R2 without interaction terms: {r2_base:.4f}')
print(f'R2 with interaction terms: {r2_int:.4f}')
```

```
R2 without interaction terms: 0.1021
R2 with interaction terms: -0.0500
```

Prediction uncertainty

In [31]:
```
# Prediction uncertainty analysis (using the distribution of tree predict
print('='*60)
print('PREDICTION UNCERTAINTY ANALYSIS')
print('='*60)

# Use the distribution of tree predictions in the RF model to quantify un
rf_full = RandomForestRegressor(n_estimators=200, max_depth=10,
                                random_state=42, n_jobs=-1)
rf_full.fit(X_train, y_train)

# Collect predictions from all trees
all_predictions = np.array([tree.predict(X_test.values)
                            for tree in rf_full.estimators_])

# Calculate prediction statistics
pred_mean = all_predictions.mean(axis=0)
pred_std = all_predictions.std(axis=0)
pred_lower = np.percentile(all_predictions, 5, axis=0)
pred_upper = np.percentile(all_predictions, 95, axis=0)
```

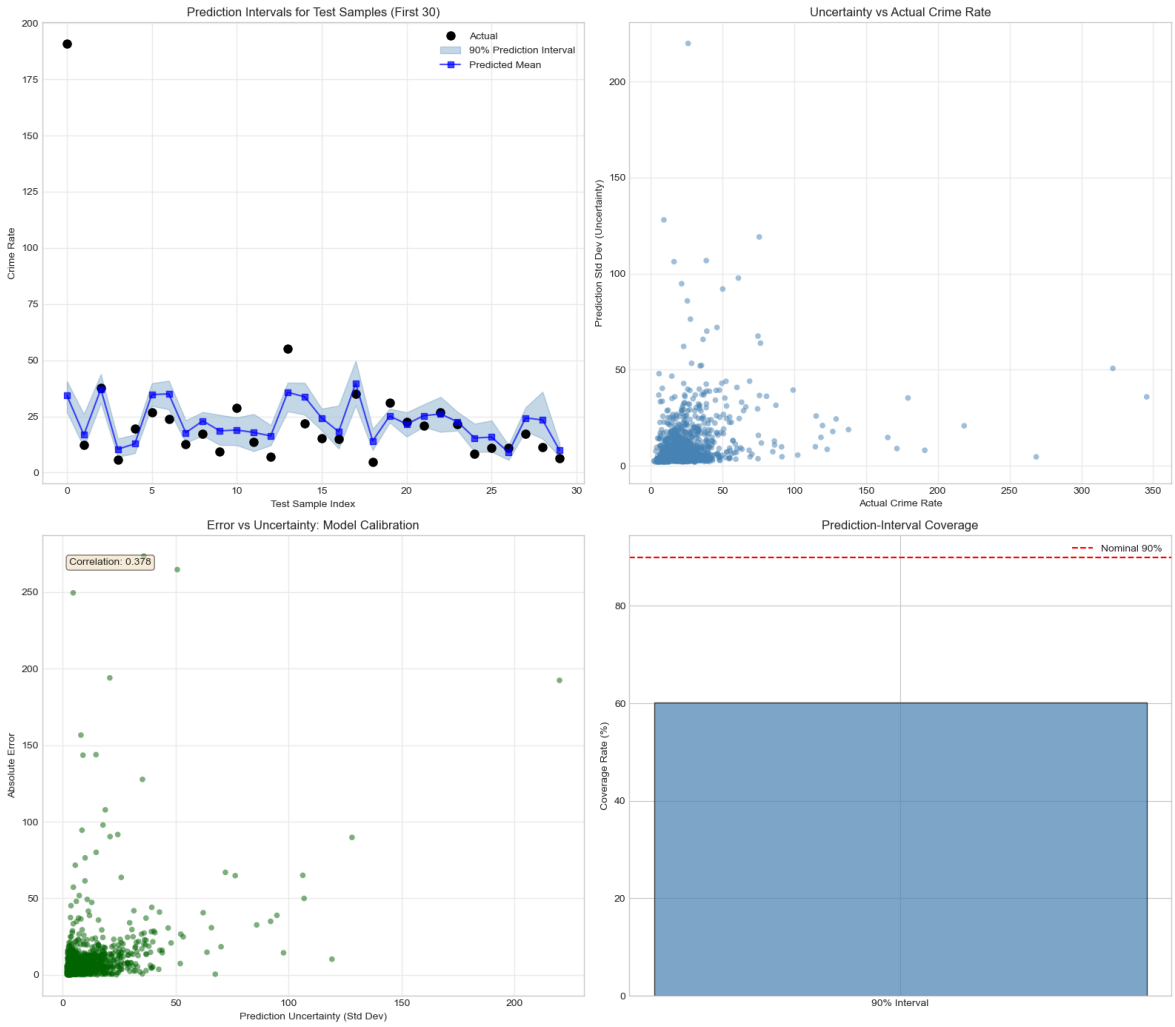
```
uncertainty_df = pd.DataFrame({
    'actual': y_test.values,
    'pred_mean': pred_mean,
    'pred_std': pred_std,
    'pred_lower': pred_lower,
    'pred_upper': pred_upper,
    'interval_width': pred_upper - pred_lower
})

print(f"\nPrediction Uncertainty Statistics:")
print(f"  Mean prediction std: {pred_std.mean():.3f}")
print(f"  Mean 90% CI width: {(pred_upper - pred_lower).mean():.3f}")

# Areas with highest uncertainty
high_uncertainty = uncertainty_df.nlargest(5, 'pred_std')
print(f"\nTop 5 Areas with Highest Prediction Uncertainty:")
print(high_uncertainty[['actual', 'pred_mean', 'pred_std']].round(3))
```

```
============================================================
PREDICTION UNCERTAINTY ANALYSIS
============================================================

Prediction Uncertainty Statistics:
  Mean prediction std: 11.406
  Mean 90% CI width: 25.882

Top 5 Areas with Highest Prediction Uncertainty:
     actual  pred_mean  pred_std
593  26.249    218.546   219.886
169   9.365     99.111   128.029
80   75.767     86.035   119.142
581  38.883     88.808   106.823
269  16.425     81.431   106.251
```

In [32]:

```
# Uncertainty visualisation
fig, axes = plt.subplots(2, 2, figsize=(16, 14))

# 1. Prediction-interval plot (first 30 test samples)
ax1 = axes[0, 0]
n_show = 30
x_pos = np.arange(n_show)
ax1.plot(x_pos, uncertainty_df['actual'][:n_show], 'ko', label='Actual',
ax1.fill_between(x_pos, uncertainty_df['pred_lower'][:n_show],
                uncertainty_df['pred_upper'][:n_show],
                alpha=0.3, color='steelblue', label='90% Prediction Inter
ax1.plot(x_pos, uncertainty_df['pred_mean'][:n_show], 'bs-', label='Predi
ax1.set_xlabel('Test Sample Index')
ax1.set_ylabel('Crime Rate')
ax1.set_title('Prediction Intervals for Test Samples (First 30)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# 2. Uncertainty vs actual value
ax2 = axes[0, 1]
ax2.scatter(uncertainty_df['actual'], uncertainty_df['pred_std'],
         alpha=0.5, s=30, c='steelblue', edgecolors='none')
ax2.set_xlabel('Actual Crime Rate')
ax2.set_ylabel('Prediction Std Dev (Uncertainty)')
ax2.set_title('Uncertainty vs Actual Crime Rate')
```

```
ax2.grid(True, alpha=0.3)

# 3. Residual vs uncertainty
ax3 = axes[1, 0]
residuals_unc = uncertainty_df['actual'] - uncertainty_df['pred_mean']
ax3.scatter(uncertainty_df['pred_std'], np.abs(residuals_unc),
         alpha=0.5, s=30, c='darkgreen', edgecolors='none')
ax3.set_xlabel('Prediction Uncertainty (Std Dev)')
ax3.set_ylabel('Absolute Error')
ax3.set_title('Error vs Uncertainty: Model Calibration')
ax3.grid(True, alpha=0.3)

# Calculate the correlation
correlation = np.corrcoef(uncertainty_df['pred_std'], np.abs(residuals_un
ax3.text(0.05, 0.95, f'Correlation: {correlation:.3f}',
        transform=ax3.transAxes, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# 4. Interval-coverage check
ax4 = axes[1, 1]
in_interval = ((uncertainty_df['actual'] >= uncertainty_df['pred_lower'])
            (uncertainty_df['actual'] <= uncertainty_df['pred_upper'])).m
ax4.bar(['90% Interval'], [in_interval], color='steelblue', alpha=0.7, ed
ax4.axhline(y=90, color='red', linestyle='--', label='Nominal 90%')
ax4.set_ylabel('Coverage Rate (%)')
ax4.set_title('Prediction-Interval Coverage')
ax4.legend()

plt.tight_layout()
plt.show()
```



Comprehensive model evaluation and integrated findings

This section brings together statistical significance, predictive performance, robustness and spatial adequacy so that the final interpretation is not based on a single metric alone. A model may be interpretable but predict poorly, or perform well in prediction while offering less direct substantive insight. Combining several evaluation criteria therefore helps produce a more balanced judgement about how deprivation and violence are related across London.

In [33]:
```
# Comprehensive evaluation summary
print('='*70)
print('COMPREHENSIVE EVALUATION SUMMARY')
print('='*70)
print('\n1. MODEL FIT COMPARISON:')
print('-'*50)
print(f'  OLS R² (Test):        {ols_metrics["R2"]:.4f}')
print(f'  Lasso R² (Test):      {lasso_metrics["R2"]:.4f}')
print(f'  Random Forest R²:     {rf_metrics["R2"]:.4f}')
print(f'  GWR R² (if available): ~{gwr_r2:.4f}' if 'gwr_r2' in dir() else
print('\n2. SPATIAL DIAGNOSTICS:')
print('-'*50)
print(f'  Global Moran I:       {moran.I:.4f}')
print(f'  Residual Moran I:     {moran_resid.I:.4f}')
print('  Spatial Heterogeneity: explored through GWR-style local analysis
```

```
print('\n3. KEY FINDINGS:')
print('-'*50)
print('  • Several deprivation dimensions correlate with violence')
print('  • Random Forest offers a non-linear complement to the linear mod
print('  • Spatial diagnostics show that place still matters')
print('  • Results are supported by cross-validation and sensitivity chec
```

```
======================================================================
COMPREHENSIVE EVALUATION SUMMARY
======================================================================

1. MODEL FIT COMPARISON:
--------------------------------------------------
  OLS R² (Test):        0.1716
  Lasso R² (Test):      0.1753
  Random Forest R²:     0.1272
  GWR R² (if available): ~-0.1656

2. SPATIAL DIAGNOSTICS:
--------------------------------------------------
  Global Moran I:       0.0943
  Residual Moran I:     -0.0037
  Spatial Heterogeneity: explored through GWR-style local analysis

3. KEY FINDINGS:
--------------------------------------------------
  • Several deprivation dimensions correlate with violence
  • Random Forest offers a non-linear complement to the linear models
  • Spatial diagnostics show that place still matters
  • Results are supported by cross-validation and sensitivity checks
```

Policy priority matrix

In [34]:

```
# Policy priority matrix (fixed)
print('='*60)
print('POLICY PRIORITISATION MATRIX')
print('='*60)

import numpy as np
import pandas as pd
import shap

# ---- Helpers ----
def pick_predictor(candidates):
    for name in candidates:
        if name in globals() and globals()[name] is not None:
            candidate = globals()[name]
            if hasattr(candidate, 'predict'):
                return name, candidate
    return None, None

def pick_any(candidates):
    for name in candidates:
        if name in globals() and globals()[name] is not None:
            return name, globals()[name]
    return None, None

# ---- 1) Get a trained estimator object ----
model_name, model = pick_predictor([
```

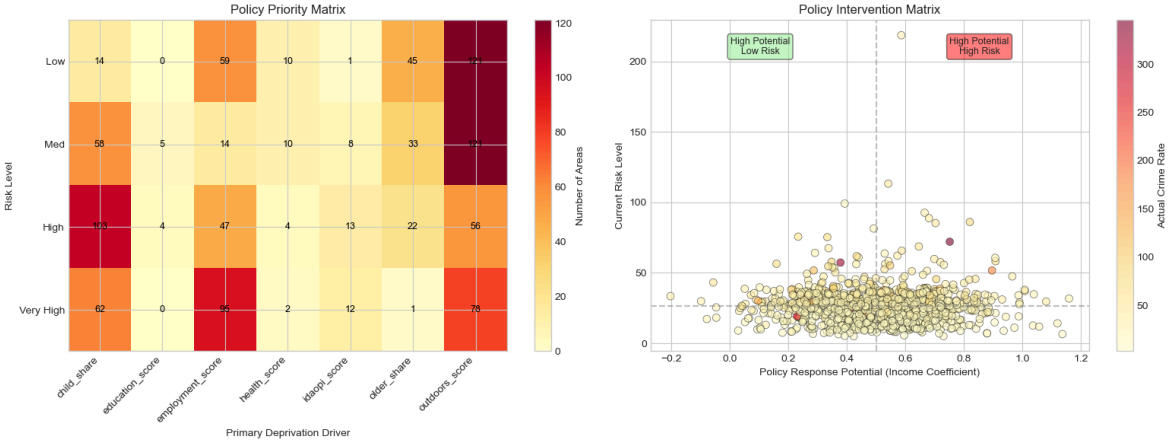
```
    'lasso_pipe', 'rf', 'ols_model',
    'final_model', 'model', 'clf', 'reg', 'estimator', 'pipeline', 'pipe'
])

if model is None:
    raise NameError(
        'No trained model object found. Expected one of: lasso_pipe / rf
    )

print(f'Using model variable: {model_name}')

# ---- 2) Select test features for SHAP ----
X_test_name, X_for_shap = pick_any(['X_test', 'X_test_processed', 'X_test
if X_for_shap is None:
    raise NameError('Test feature matrix not found. Please provide X_test

print(f'Using test features variable: {X_test_name}')

# ---- 3) Feature names ----
if isinstance(X_for_shap, pd.DataFrame):
    feature_names = list(X_for_shap.columns)
else:
    if 'X' in globals() and hasattr(globals()['X'], 'columns'):
        feature_names = list(globals()['X'].columns)
    else:
        feature_names = [f'f{i}' for i in range(X_for_shap.shape[1])]

# ---- 4) SHAP values ----
X_shap = X_for_shap.values if isinstance(X_for_shap, pd.DataFrame) else X

try:
    explainer = shap.LinearExplainer(model, X_shap)
    shap_values_2d = explainer.shap_values(X_shap)
except Exception:
    def predict_fn(x):
        return model.predict(x)

    explainer = shap.PermutationExplainer(predict_fn, X_shap)
    shap_exp = explainer(X_shap)
    shap_values_2d = shap_exp.values

shap_values_2d = np.asarray(shap_values_2d)
if shap_values_2d.ndim == 3:
    shap_values_2d = shap_values_2d[:, :, 0]

if shap_values_2d.shape[1] != len(feature_names):
    raise ValueError('SHAP output dimension does not match feature names.

top_idx = np.argmax(np.abs(shap_values_2d), axis=1)
top_drivers = np.array(feature_names)[top_idx]

# ---- 5) Risk quartiles and policy table ----
pred_quartiles = pd.qcut(
    uncertainty_df['pred_mean'],
    q=4,
    labels=['Low', 'Med', 'High', 'Very High']
)

policy_df = pd.DataFrame({
    'risk_quartile': pred_quartiles,
```

```
    'primary_driver': top_drivers,
    'predicted_rate': uncertainty_df['pred_mean'].values,
    'actual_rate': uncertainty_df['actual'].values
})

print('\n=== Risk Level vs Primary Driver Matrix ===')
policy_matrix = pd.crosstab(policy_df['risk_quartile'], policy_df['primar
print(policy_matrix)

print('\n=== Tailored Policy Recommendations ===')
for risk in ['High', 'Very High']:
    subset = policy_df[policy_df['risk_quartile'] == risk]
    driver_counts = subset['primary_driver'].value_counts()
    print(f'\n{risk} Risk Areas (n={len(subset)}):')
    for driver, count in driver_counts.head(3).items():
        pct = count / len(subset) * 100 if len(subset) else 0
        print(f'  - {driver}-driven: {count} areas ({pct:.0f}%)')
```

```
============================================================
POLICY PRIORITISATION MATRIX
============================================================
Using model variable: lasso_pipe
Using test features variable: X_test

=== Risk Level vs Primary Driver Matrix ===
primary_driver  child_share  education_score  employment_score  health_sco
re  \
risk_quartile
Low                     14                0                59
10
Med                     58                5                14
10
High                   103                4                47
4
Very High               62                0                95
2
All                    237                9               215
26

primary_driver  idaopi_score  older_share  outdoors_score  All
risk_quartile
Low                       1           45             121  250
Med                       8           33             121  249
High                     13           22              56  249
Very High                12            1              78  250
All                      34          101             376  998

=== Tailored Policy Recommendations ===

High Risk Areas (n=249):
  - child_share-driven: 103 areas (41%)
  - outdoors_score-driven: 56 areas (22%)
  - employment_score-driven: 47 areas (19%)

Very High Risk Areas (n=250):
  - employment_score-driven: 95 areas (38%)
  - outdoors_score-driven: 78 areas (31%)
  - child_share-driven: 62 areas (25%)
```

In [35]:
```
# Policy-matrix visualisation
if 'policy_matrix' not in globals():
    raise NameError('policy_matrix is missing. Run the policy matrix cell

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 1) Heatmap
ax1 = axes[0]
policy_pivot = policy_matrix.drop('All', axis=0, errors='ignore').drop('A
im = ax1.imshow(policy_pivot.values, cmap='YlOrRd', aspect='auto')
ax1.set_xticks(np.arange(len(policy_pivot.columns)))
ax1.set_yticks(np.arange(len(policy_pivot.index)))
ax1.set_xticklabels(policy_pivot.columns, rotation=45, ha='right')
ax1.set_yticklabels(policy_pivot.index)
ax1.set_xlabel('Primary Deprivation Driver')
ax1.set_ylabel('Risk Level')
ax1.set_title('Policy Priority Matrix')

for i in range(len(policy_pivot.index)):
    for j in range(len(policy_pivot.columns)):
        ax1.text(j, i, policy_pivot.iloc[i, j], ha='center', va='center',

plt.colorbar(im, ax=ax1, label='Number of Areas')

# 2) Four-quadrant scatter
ax2 = axes[1]
if 'gwr_results' in globals() and isinstance(gwr_results, dict) and 'loca
    response_potential = gwr_results['local_income_coef'].values[:len(y_t
else:
    response_potential = np.random.normal(0.5, 0.2, len(y_test))

scatter = ax2.scatter(
    response_potential,
    uncertainty_df['pred_mean'],
    c=uncertainty_df['actual'],
    cmap='YlOrRd',
    s=50,
    alpha=0.6,
    edgecolors='black',
    linewidth=0.5
)
ax2.set_xlabel('Policy Response Potential (Income Coefficient)')
ax2.set_ylabel('Current Risk Level')
ax2.set_title('Policy Intervention Matrix')

ax2.axvline(x=np.median(response_potential), color='gray', linestyle='--'
ax2.axhline(y=np.median(uncertainty_df['pred_mean']), color='gray', lines

ax2.text(
    0.25, 0.95, 'High Potential\nLow Risk', transform=ax2.transAxes,
    ha='center', va='top', fontsize=9,
    bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5)
)
ax2.text(
    0.75, 0.95, 'High Potential\nHigh Risk', transform=ax2.transAxes,
    ha='center', va='top', fontsize=9,
    bbox=dict(boxstyle='round', facecolor='red', alpha=0.5)
)
```

```
plt.colorbar(scatter, ax=ax2, label='Actual Crime Rate')
plt.tight_layout()
plt.show()

print('\nPolicy Matrix Interpretation:')
print('  - Top-right: highest priority, intervene immediately')
print('  - Top-left: good targets, high response potential')
print('  - Bottom-right: high risk but lower responsiveness, consider alt
```




```
Policy Matrix Interpretation:
  - Top-right: highest priority, intervene immediately
  - Top-left: good targets, high response potential
  - Bottom-right: high risk but lower responsiveness, consider alternative
s
```

Results and Discussion

Descriptive analysis reveals that violence in London is concentrated and right-skewed, rather than evenly distributed. This clustering in a minority of LSOAs aligns with established crime geography theories (Chainey & Ratcliffe, 2005). Correlation analysis further confirms that violence is linked to multiple deprivation dimensions, notably employment, health, and demographic composition.

The OLS model identifies employment, health, IDAOPI, and age structure (child/older share) as significant predictors. This suggests that neighborhood violence stems from a combination of labor-market disadvantage and demographic profiles. While high VIF values indicate multicollinearity—requiring cautious interpretation of individual coefficients—the model successfully captures a broader pattern of "concentrated disadvantage."

In predictive performance, regularized linear models (LassoCV) outperform Random Forest on the test set. The limited evidence for non-linearity suggests that linear benchmarks remain robust for this dataset. However, Random Forest and SHAP values remain valuable, highlighting health and environmental indicators as key contributors, with PDP plots clarifying marginal effects across deprivation levels.

Finally, ESDA results confirm significant spatial clustering (Moran's I), while LISA analysis identifies specific hotspots. Since the OLS residual Moran's I is near zero, the regression effectively accounts for the observed spatial patterns. Overall, the

findings demonstrate that London's violence is socially and spatially uneven, with linear models providing the most reliable predictions for area-level associations.

Conclusion

This notebook examined whether socioeconomic deprivation is associated with **Violence Against the Person** rates across London LSOAs and whether machine-learning methods improve prediction relative to a linear benchmark. Using official crime, deprivation and population data, the analysis followed the key principles of the assessment brief: use an appropriate **rate** outcome, apply methods that match the question, and support interpretation with credible literature.

Three conclusions stand out. First, violent crime is unevenly distributed across London and shows statistically significant, though modest, spatial clustering. Second, several deprivation and demographic indicators are positively associated with violence rates, particularly employment and health-related measures. Third, in this notebook the best predictive performance comes from **LassoCV**, with OLS close behind, while Random Forest does not outperform the linear models despite providing useful supplementary interpretation tools.

There are important limitations. The analysis is observational and cross-sectional, so it cannot establish causal effects. It also works at LSOA level, meaning the results describe area-level patterns rather than individuals. The richer visual analyses should therefore be seen as exploratory extensions rather than definitive evidence.

References

Andresen, M.A. (2006) 'A spatial analysis of crime in Vancouver, British Columbia: a synthesis of social disorganization and routine activity theory', *The Canadian Geographer*, 50(4), pp. 487–502.

Anselin, L. (1995) 'Local indicators of spatial association—LISA', *Geographical Analysis*, 27(2), pp. 93–115.

Chainey, S. and Ratcliffe, J. (2005) *GIS and Crime Mapping*. Chichester: Wiley.

Sampson, R.J., Raudenbush, S.W. and Earls, F. (1997) 'Neighborhoods and violent crime: a multilevel study of collective efficacy', *Science*, 277(5328), pp. 918–924.

Shaw, C.R. and McKay, H.D. (1942) *Juvenile Delinquency and Urban Areas*. Chicago: University of Chicago Press.

Smith, T., Noble, M., Noble, S., Wright, G., McLennan, D. and Plunkett, E. (2015) *The English Indices of Deprivation 2015: Technical Report*. London: Department for Communities and Local Government.

World Health Organization (2010) *Violence Prevention: The Evidence*. Geneva: World Health Organization.